

Chapter 5... Pointers

5.1 Introduction to Pointers

Pointer Definition

A **pointer** in C is a variable that stores the **memory address** of another variable instead of storing the actual value.

Explanation of Pointer

1. A pointer is a **special variable** used to store the address of another variable.
2. It is declared using the **asterisk (*) symbol**.
Example: `int *p;`
3. The pointer stores the **memory location (address)** of a variable.
4. The **address of a variable** is obtained using the **address operator (&)**.
Example: `p = &a;`
5. The **value stored at the address** can be accessed using the **dereference operator (*)**.
6. Pointers help in **direct memory access**, making programs more efficient.
7. They are widely used in **arrays, functions, and dynamic memory allocation**.
8. Pointers allow passing arguments by **reference**, enabling modification of original variables.

Advantages of Pointers

1. **Efficient memory usage** – avoids duplication of variables.
2. **Fast execution** – direct access to memory improves performance.
3. **Dynamic memory allocation** – used with functions like `malloc()` and `free()`.
4. **Useful in data structures** – like linked lists, stacks, and queues.

Example:

```
#include <stdio.h>
```

```
#include <conio.h>

void main()
{
    int a = 10;
    int *p;

    clrscr(); // clear screen
    p = &a;   // store address of variable a

    printf("Value of a = %d\n", a);
    printf("Address of a = %u\n", &a);
    printf("Pointer p stores = %u\n", p);
    printf("Value using pointer = %d\n", *p);

    getch(); // wait for key press
}
```

Expected Output

```
Value of a = 10
Address of a = 65524 (may vary)
Pointer p stores = 65524
Value using pointer = 10
```

5.1.2 Pointer Fundamentals in C

Definition

A pointer is a variable that stores the **address of another variable**.

Explanation

1. A pointer is a **variable that holds memory address** of another variable.
2. Every variable in C has a **unique address in memory**.
3. The **address operator (&)** is used to get the address of a variable.
Example: &a
4. A pointer is declared using the **asterisk (*) symbol**.
Example: int *p;

5. A pointer must be **initialized** before use by assigning address.
Example: `p = &a;`
6. The **dereference operator (*)** is used to access value stored at the address.
7. Pointer and variable **share the same data indirectly**.
8. Changing value using pointer **affects the original variable**.

Example Using Address of Variable

```
#include <stdio.h>
#include <conio.h>

void main()
{
    int a = 10;
    int *p;

    clrscr();

    p = &a; // store address of a

    printf("Value of a = %d\n", a);
    printf("Address of a = %u\n", &a);
    printf("Pointer p = %u\n", p);
    printf("Value using pointer = %d\n", *p);

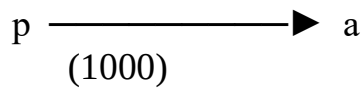
    getch();
}
```

Diagram (Memory Representation)

Variable	Value	Address

a	10	1000
p	1000	2000

Explanation:



`a = 10`

- `a` is stored at address **1000**
- `p` stores **1000 (address of a)**
- `*p` gives value **10**

5.1.3 Pointer Declaration / Notations

Definition

Pointer declaration means defining a pointer variable using `*` so that it can store the **address of a variable**.

Explanation of Pointer Declaration / Notations

1. A pointer is declared using the **asterisk (*) symbol** before the variable name.
2. Syntax of pointer declaration:
3. `data_type *pointer_name;`
4. The **data type of pointer** must match the data type of variable whose address it stores.
5. Example:
6. `int *p;`
7. `float *q;`
8. Multiple pointers can be declared in a single statement:
9. `int *p, *q;`
10. Pointer can store address using **address operator (&)**.
11. `p = &a;`
12. `*` is used for **declaration as well as dereferencing**.
13. Pointer variables should be **initialized before use** to avoid errors.

5.1 A program contains the following pointer declarations

```
float a,b    int p,q        char k        float *pa;
```

5.2 Example Program Different Pointer Declarations

```
#include <stdio.h>
#include <conio.h>

void main()
{
    int a = 10, b = 20;
    float x = 5.5, y = 7.5;
    char ch = 'A';

    int *p1, *p2;    // integer pointers
    float *f1, *f2;  // float pointers
    char *cptr;      // char pointer

    clrscr();

    /* Assigning addresses */
    p1 = &a;
    p2 = &b;

    f1 = &x;
    f2 = &y;

    cptr = &ch;

    /* Printing values using pointers */
    printf("a = %d, b = %d\n", *p1, *p2);
    printf("x = %f, y = %f\n", *f1, *f2);
    printf("ch = %c\n", *cptr);

    getch();
}
```

5.3.1.1 difference between '*' and '&' operators

& (Address Operator)

1. & is used to **get the address of a variable**.

2. It returns the **memory location**.
3. Used during **pointer initialization**.
4. Example:
5. `int a = 10;`
6. `int *p;`
7. `p = &a; // & gives address of a`

* (Dereference Operator)

1. * is used to **get value stored at an address**.
2. It returns the **actual value**.
3. Used to **access or modify value using pointer**.
4. Example:
5. `int a = 10;`
6. `int *p;`
7. `p = &a;`
8. `printf("%d", *p); // * gives value of a`

5.1.4 Pointer Initialization

Definition

Pointer initialization means **assigning a valid memory address** to a pointer variable.

Points

1. After a pointer is declared, but before it is assigned, it contains a **garbage (unknown) value**.
2. Such a pointer does not point to any **valid memory location**.
3. Using an uninitialized pointer may cause **errors or crashes**.
4. A pointer can be initialized using the **address of a variable (&)**.
5. A pointer can also be assigned **NULL** to indicate it points to nothing.
6. **NULL is used because C guarantees that no valid object exists at NULL address.**

Example

```
#include <stdio.h>
int main()
{
    int a = 10, *p = &a;
    printf("%d", *p);
}
```

Example Using NULL

```
int *p = NULL; // pointer initialized with NULL
```

5.9 Program: Addition of Two Numbers Using Pointer

Program : To perform addition of two numbers using pointer variables.

```
#include <stdio.h>
#include <conio.h>

void main()
{
    int a = 5, b = 7, sum;
    int *p1, *p2;

    clrscr();

    p1 = &a; // pointer to a
    p2 = &b; // pointer to b

    sum = *p1 + *p2; // addition using pointers

    printf("Sum = %d", sum);

    getch();
}
```

Output: Sum = 12

Explanation p1 and p2 store addresses of a and b.

1. *p1 gives value of a, *p2 gives value of b.
2. Addition is done using dereferenced values.
3. Result is stored in sum.

5.10 Program: Product of Two Numbers Using Pointer

Program : To find the product of two numbers using pointer variables.

```
#include <stdio.h>
#include <conio.h>

void main()
{
    int a = 4, b = 5, product;
    int *p1, *p2;

    clrscr();

    p1 = &a; // pointer to a
    p2 = &b; // pointer to b

    product = (*p1) * (*p2); // multiplication using pointers

    printf("Product = %d", product);

    getch();
}
```

Output

Product = 20

Explanation p1 and p2 store addresses of a and b.

1. *p1 gives value of a, *p2 gives value of b.
2. Multiplication is done using dereferenced values.
3. Result is stored in product.

5.2 Pointer Arithmetic

Types of Pointer Arithmetic

1. Increment (p++)
2. Decrement (p--)
3. Addition (p + n)
4. Subtraction (p - n)

1. Increment Pointer (p++)

1. Incrementing a pointer increases it by **size of data type**.
2. It moves pointer to the **next memory location**.

Example

```
#include <stdio.h>
#include <conio.h>

void main()
{
    int number = 50;
    int *p;

    clrscr();

    p = &number;

    printf("Address before increment = %u\n", p);
```

```

p++; // increment pointer

printf("Address after increment = %u\n", p);

getch();
}

```

2. Decrement Pointer (p--)

1. Decrementing a pointer decreases it by **size of data type**.
2. It moves pointer to the **previous memory location**.

Example

```

#include <stdio.h>
#include <conio.h>

void main()
{
    int number = 50;
    int *p;

    clrscr();

    p = &number;

    printf("Address before decrement = %u\n", p);

    p--; // decrement pointer

    printf("Address after decrement = %u\n", p);

    getch();
}

```

3. Addition of Pointer (p + n)

1. Adding a number to pointer moves it **forward by n elements**.
2. Actual movement depends on **data type size**.

Example

```
#include <stdio.h>
#include <conio.h>

void main()
{
    int number = 50;
    int *p;

    clrscr();

    p = &number;

    printf("Address before addition = %u\n", p);

    p = p + 3; // add 3 to pointer

    printf("Address after addition = %u\n", p);

    getch();
}
```

4. Subtraction of Pointer (p - n)

1. Subtracting a number from pointer moves it **backward by n elements**.
2. Used to access **previous memory locations**.

Example

```
#include <stdio.h>
#include <conio.h>

void main()
{
    int number = 50;
    int *p;

    clrscr();

    p = &number;
```

```

printf("Address before subtraction = %u\n", p);

p = p - 3; // subtract 3

printf("Address after subtraction = %u\n", p);

getch();
}

```

5.3 Pointer and One-Dimensional Array

Definition

In C, the name of an array acts like a **pointer to its first element**, so pointers can be used to access array elements.

Explanation

1. The **array name represents the base address** (address of first element).
2. A pointer can be assigned to an array using:
3. `p = arr;`
4. Array elements can be accessed using **pointer arithmetic**.
5. `*(p + i)` is equivalent to `arr[i]`.
6. Pointer increases by **size of data type** when incremented.
7. Arrays and pointers are **closely related in C**.
8. Using pointers, we can **traverse array efficiently**.
9. Pointers reduce complexity when working with **large arrays**.

5.18 Program: Print Memory Location of Array Elements

```

#include <stdio.h>
#include <conio.h>

void main()
{
    int num[] = {24, 34, 12, 44, 56, 17};
    int *p;

```

```

int i;

clrscr();

p = num; // base address of array

for(i = 0; i < 6; i++)
{
    printf("Value = %d\tAddress = %u\n", *(p + i), (p + i));
}

getch();
}

```

Output

```

Value = 24   Address = 1000
Value = 34   Address = 1002
Value = 12   Address = 1004
Value = 44   Address = 1006
Value = 56   Address = 1008
Value = 17   Address = 1010

```

5.3.1 Array of Pointers

Definition

An array of pointers is an array where each element is a **pointer variable that stores addresses**.

Explanation

1. An array of pointers stores **addresses of variables** instead of actual values.
2. It is declared as:
3. `data_type *arr[size];`
4. Each element of the array points to a **separate variable or memory location**.

5. It is useful for handling **multiple strings, arrays, or dynamic data efficiently**.

Example:

```
#include <stdio.h>
#include <conio.h>

void main()
{
    int a = 10, b = 20, c = 30;
    int *p[3]; // array of pointers

    clrscr();

    p[0] = &a;
    p[1] = &b;
    p[2] = &c;

    printf("%d %d %d", *p[0], *p[1], *p[2]);

    getch();
}
```

5.22 Program: Take Data and Print Using Pointer

Program : To accept elements in an array and print them using pointer.

```
#include <stdio.h>
#include <conio.h>

void main()
{
    int num[5];
    int *p;
    int i;

    clrscr();
```

```

p = num; // pointer to array

printf("Enter 5 elements:\n");

for(i = 0; i < 5; i++)
{
    scanf("%d", (p + i)); // input using pointer
}

printf("Elements are:\n");

for(i = 0; i < 5; i++)
{
    printf("%d\n", *(p + i)); // output using pointer
}

getch();
}

```

Explanation

1. `p = num;` → pointer stores base address of array
2. `(p + i)` → gives address of each element
3. `*(p + i)` → accesses value of each element
4. Input and output done using pointer

5.23 Program: Find Length of String without Inbuilt Function

Program: To find the length of a string without using any inbuilt function.

```

#include <stdio.h>
#include <conio.h>

void main()
{
    char str[50];
    int length = 0;
    char *p;

    clrscr();

```

```

printf("Enter a string: ");
gets(str);

p = str; // pointer to string

while(*p != '\0') // loop till end of string
{
    length++;
    p++; // move pointer to next character
}

printf("Length of string = %d", length);

getch();
}

```

Explanation

p = str; → pointer points to first character

1. *p != '\0' → checks end of string
2. p++ → moves to next character
3. Length is counted without using strlen()

Output (Example)

```

Enter a string: Hello
Length of string = 5

```

5.24 Program: Copy String without strcpy()

Program : To copy one string into another without using any inbuilt string function.

```

#include <stdio.h>
#include <conio.h>

```

```

void main()
{
    char str1[50], str2[50];
    char *p1, *p2;

    clrscr();

    printf("Enter a string: ");
    gets(str1);

    p1 = str1; // pointer to source string
    p2 = str2; // pointer to destination string

    while(*p1 != '\0')
    {
        *p2 = *p1; // copy character
        p1++;
        p2++;
    }

    *p2 = '\0'; // end of string

    printf("Copied string = %s", str2);

    getch();
}

```

Explanation

p1 points to source string, p2 to destination.

1. Characters are copied one by one using `*p2 = *p1`.
2. Loop runs until `'\0'` (end of string).
3. `'\0'` is added at end of new string.

Output (Example)

Enter a string: Hello
 Copied string = Hello

5.25 Program: Display String in Reverse using Pointer

Program : To display a string in reverse order using pointer.

```
#include <stdio.h>
#include <conio.h>

void main()
{
    char str[50];
    char *p;
    int length = 0, i;

    clrscr();

    printf("Enter a string: ");
    gets(str);

    p = str;

    /* Find length of string */
    while(*(p + length) != '\0')
    {
        length++;
    }

    printf("String in reverse order:\n");

    /* Print in reverse using pointer */
    for(i = length - 1; i >= 0; i--)
    {
        printf("%c", *(p + i));
    }

    getch();
}
```

```
}
```

Explanation

Pointer p stores base address of string.

1. Length is calculated using pointer.
2. Loop runs from last character to first.
3. `*(p + i)` is used to print characters in reverse.

Output (Example)

Enter a string: Hello
String in reverse order:
olleH

5.4 Handling Function using Pointer

Definition

A function pointer is a pointer that stores the **address of a function** and is used to call the function indirectly.

Syntax (Declaration)

```
return_type (*pointer_name)(parameter_list);
```

```
int (*fptr)(int, int);
```

Pointer Declaration

1. A function pointer is declared using `(*pointer_name)` syntax.
2. The return type of pointer must match the **function return type**.
3. The parameters of pointer must match the **function parameters**.
4. Parentheses `()` are necessary to differentiate from normal pointer.

Reading Function Pointer Declaration

`int (*fptr)(int, int) → fptr` is a pointer to a function taking two integers and returning `int`.

1. It means `fptr` can store the **address of such functions**.

Initializing Function Pointer

```
fptr = add; // assign function address
```

OR

```
fptr = &add;
```

Using Function Pointer

```
result = fptr(10, 20); // calling function using pointer
```

Example

```
#include <stdio.h>
#include <conio.h>
```

```
int add(int a, int b)
{
    return a + b;
}
```

```
void main()
{
    int (*fptr)(int, int);
    int result;
```

```
    clrscr();
```

```
    fptr = add; // initialize pointer
```

```
    result = fptr(5, 3); // call function using pointer
```

```
    printf("Sum = %d", result);
```

```
    getch();  
}
```

- Function pointer stores **address of function**
- Used to call function indirectly
- Useful for **flexible and dynamic programming**

5.5 Handling Structure using Pointer

A structure pointer is a pointer that stores the **address of a structure variable** and is used to access its members.

1. A pointer can be used to **access members of a structure using address**.
2. Structure pointers are mostly used with **array of structures** for efficient data handling.

Structure Declaration

```
struct book  
{  
    char name[10];  
    int price;  
};
```

Program

```
#include <stdio.h>  
#include <conio.h>
```

```
struct book  
{  
    char name[10];  
    int price;  
};
```

```
void main()  
{  
    struct book b1 = {"CBook", 250};  
    struct book *p;
```

```

clrscr();

p = &b1; // pointer to structure

printf("Book Name = %s\n", p->name);
printf("Book Price = %d", p->price);

getch();
}

```

Explanation

p = &b1; → pointer stores address of structure

- p->name → access structure member using pointer
- -> operator is used with structure pointer

Points (Accessing Structure Members)

1. A **structure** is a collection of different data types grouped together.
2. Members of a structure are accessed using the **dot (.) operator**.
3. Structure variables must be **declared before use**.
4. Values can be assigned and accessed using variable.member.

Program

```

#include <stdio.h>
#include <conio.h>

struct student
{
    char name[20];
    int number;
    int rank;
};

void main()
{
    struct student s;

    clrscr();

    /* Assign values */

```

```
strcpy(s.name, "studytonight");
s.number = 35;
s.rank = 1;

/* Display values */
printf("Name: %s\n", s.name);
printf("Number: %d\n", s.number);
printf("Rank: %d\n", s.rank);

getch();
}
```

Output:

```
Name: studytonight
Number: 35
Rank: 1
```